Amrita Vishwa Vidyapeetham

Real-Time Multi-Core Sorting and Median Filter Service

26ELC331 — Real-Time Operating Systems

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

October 17, 2026

d.2 Outline

Introduction
System Overview
Methodology

Results
Use Cases and Closing

PART 1

Introduction

A client continuously offloads **unsorted floating-point time-series** metrics to a server. The server

- sorts each block with a **parallel merge sort**,
- removes sensor outliers with a **rolling median**,
- and **signals completion asynchronously**.

The client then collects the median and a memory-offset descriptor — the two things the service exists to produce.

Why it is hard
The workload arrives faster than it can be handled one sample at a time, and the readings are corrupted by spikes that no average survives.

The rules
Message passing only, a client–server split, a two-part IOV transfer, a payload of at least 64 KB, and an asynchronous event.

d.5 Why a median, and not an average

A sensor reading is one number in a stream. An outlier is a reading that the sensor, or the wiring, invented.

Window	Mean	Median	Truth
5, 5, 5, 5, 5	5.0	5.0	5
5, 5, 5, 5, 50	**14.0**	**5.0**	5

One spike moves the mean by 9 V and the median not at all. That is the entire argument.

- The filter slides a window of neighbouring samples and takes the middle value.
- The window must be **odd**, so a middle value exists.
- It needs the signal in **time order** — so it cannot run on the buffer the sort has just reordered.

On our payload the median removes the injected spikes outright; what remains is the bias it leaves on a step or a ramp, which no median can avoid.

d.6 What we set out to build

Agree on the message first
Both programs share one header file. It was written before any QNX code, so the client and the server could not drift apart — and it can be tested on a PC.

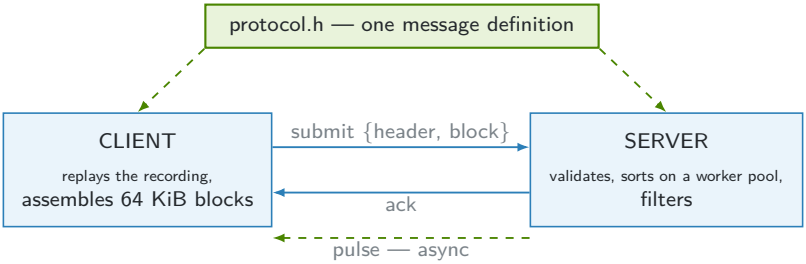
Build it in stages, test it off the board
Sensor, then loader, then sort — each finished and tested before the next starts. Almost none of the code needs QNX, so it runs on a PC first; the board is where we take the numbers, not where we debug.

- **One block, one job** — every block is exactly 64 KiB, so the size requirement needs no special case.
- **Same answer every time** — the result must not change with the number of threads, so it can be checked.
- **Show the receipts** — every number on a slide is a command run on the Raspberry Pi 4.

PART 2

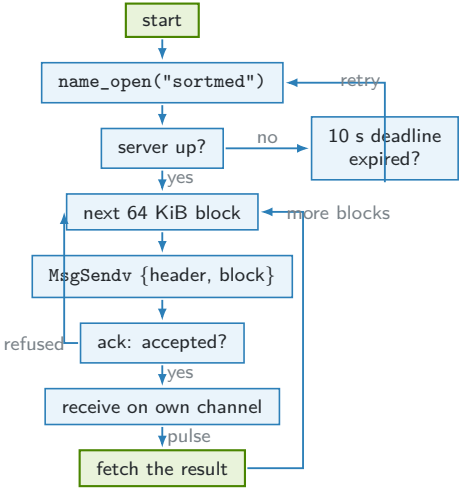
System Overview

d.8 System overview



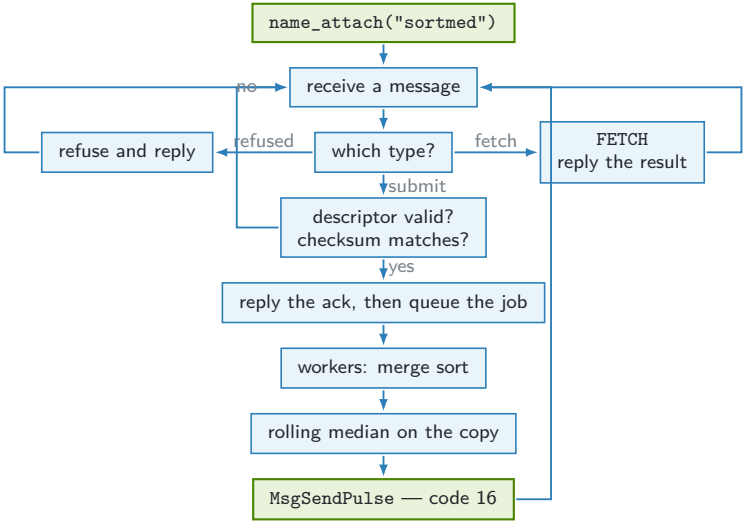
Both programs are compiled against the same header, and the 64 KiB block crosses as the second part of an IOV — the header is read, validated, and only then is the payload pulled.

d.9 Client: connect, offload, collect



no server within 10 s → report it and exit, never hang

d.10 Server: accept, validate, answer



Client — the offload side

- Connects by **name only** (`name_open`), retrying to a 10-second deadline rather than hanging.
- Replays a recording, assembles blocks, sends each as a two-part IOV.
- Owns its own connection, which is why the event can reach it later.

Server — the compute side

- `name_attach` registers the name; the receive loop never blocks on a sort.
- Validates the descriptor, verifies the checksum, replies with the ack, and only then queues the job.
- Raises the event with a pulse and answers the fetch with the median and the offset descriptor.

`The ack says "accepted"; the pulse says "done"; the fetch carries the answer`

d.12 The message definition

One header file, `protocol.h`, is compiled into both programs. The whole exchange is five messages:

Message	What it does
`SUBMIT`	header + block, one send
`ACK`	accepted, or why not
`PULSE`	sent later, when the work is done
`FETCH`	client asks for the answer
`RESULT`	the median and the offset descriptor

The block travels beside the descriptor
The server reads 48 bytes, decides, then pulls the 64 KiB. It never needs a 64 KiB buffer just to learn how big the message is.

The checksum covers only the declared samples
A short final block's tail is never checksummed, so the value cannot depend on stale buffer contents.

The client chooses the **job id** and the server echoes it in the ack, the pulse and the result, which is how a completion is matched to its request.

d.13 Implementation: the virtual sensor

- The recording is a headerless file of float32 values, so its size is its only metadata: sample count is `bytes / 4`.
- Samples are read through a **sliding window** of 4096, refilled as the reader advances — never the whole file at once.
- A read delivers at most **one frame** (128 samples), as a device would. Gathering frames into a 64 KiB block is the client's job, not the sensor's.
- The timestamp is computed, not measured: `index / rate`, so it does not drift with the host.

Nothing is assumed about the file
A missing file, an empty file, a size that is not a whole number of float32 values, a frame or rate out of range — each is refused by name, and the message says which one it was.

`test_sensor` — **15 tests, 642 checks**
Exact float round trips, frame capping, end of file, window refills at unaligned offsets, timestamps and paced reads.

- Turns the sample stream into the unit the service moves: **16 384 samples = exactly 64 KiB**, one block per read of the sensor.
- Alongside each block it fills a **descriptor**: sequence, first sample, count, rate, channel, min/max and the checksum.
- **A short final block is zero-padded.** 60 000 samples become three full blocks and one of 10 848.

Why padding matters
The whole 64 KiB crosses to another program, and uninitialised bytes should never travel. Zeroing them also makes the block reproducible.

`test_block_loader` — 8 tests, 116 checks
A synthetic 20 000-sample file splitting as 16 384 + 3616 and reassembling exactly, and a single changed sample being caught.

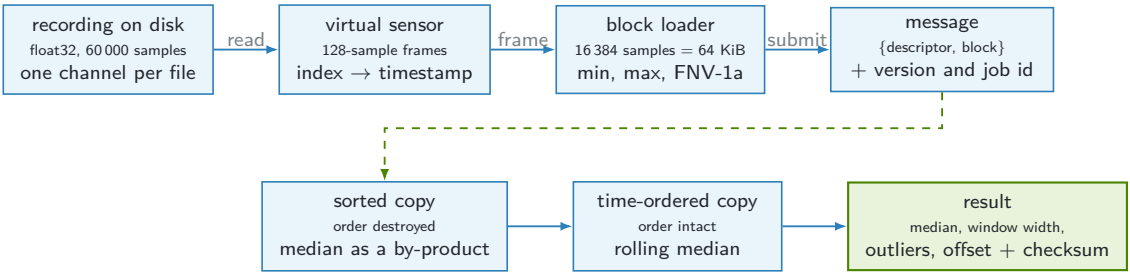
- A pool of POSIX worker threads sorts one block at a time.
- **Phase 0**: each worker sorts one partition.
- **After that**: pairs of sorted runs are merged, halving the number until one run covers the block.
- The phase boundary is a **real barrier** — one mutex and two condition variables. Nothing is ordered by a sleep.
- One scratch buffer is ping-ponged between phases: one allocation per block, none per merge.

```
/* merge: take the smaller head */
while (i < n1 && j < n2)
    out[k++] = (a[i] <= b[j]) ? a[i++] : b[
     j++];
```

`test_mergesort` — **13 tests, 578 checks**
Every size from 0 to 64, the sizes around the 4096 threshold, the partition boundaries, all-equal and reverse-sorted input, ±0.0 and the float extremes.

Equal values keep their order, so 1 worker and 8 agree exactly

d.16 Data flow: what changes at each step



Two buffers, not one. The sort destroys the time order the filter reads, so the arriving block is never sorted in place — its checksum describes it as it arrived.

Every hand-off in this chain is byte-deterministic: fixed-width fields, cleared structs, a zero-padded short block — so any two stages can be compared exactly.

d.17 One recording on disk

One channel is one file — a flat run of float32 values, no header:

File	Contents
`r1_voltage.bin`	60 000 samples, 240 000 B
`r1_current.bin`	60 000 samples, 240 000 B

- A whole file is **one signal**, so a minimum or maximum over it means something.
- The file length is the only metadata there is: 240 000 bytes is exactly 60 000 values.

Reading a sample is arithmetic
Sample `i` sits at byte `i * 4`. The loader turns that into blocks by reading a continuous run — no seeking per sample.

The channel is carried, not assumed
The blocks are labelled with which signal they hold, so a block of voltage can never be presented as current downstream.

Sorting a block with four workers

- **Phase 0** — each worker sorts one partition into the scratch buffer.
- **Merge phases** — pairs of runs are merged until one run covers the array.
- A merge reads two runs the previous phase produced, so the phase boundary is a **real barrier**: one mutex, a work-ready and a phase-done condition variable.
- Nothing is ordered by a sleep, so a slow machine cannot produce a different answer.

Deterministic by construction
The merge takes from the left run on ties, so equal values keep their order and the output is bit-identical for 1, 2, 3, 5 and 8 threads.

One allocation per sort
A single scratch buffer is ping-ponged between phases. Inputs below 4096 values take the serial path, where a pool costs more than it saves.

`4 partitions • 2 merge phases • 7 tasks`

PART 3

Methodology

THE ORDER OF WORK

1. **Protocol** — the interface both sides compile against, proved on the host before any kernel code.
2. **Sensor** — replay a recording like a device, in frames, on its own clock.
3. **Block loader** — turn the stream into 64 KiB blocks with descriptors.
4. **Merge sort** — the parallel phase machinery, measured against `qsort`.
5. **Image** — cross-compile, pack, boot, run.

Five programs, one library
`sensor.c`, `protocol.c`, `block_loader.c` and `mergesort.c` are linked into every binary — the tests and the demos exercise the same code the service will run.

Host build, target build, one source
`-std=c99 -Wall -Wextra -pedantic` clean on both. The only platform-dependent file is the sensor's clock and sleep, behind one `#if defined(_WIN32)`.

d.21 Where the client's data comes from

The workload is synthesised offline, so it can be replayed at realistic volume with a **known ground truth**.

- Three PyBaMM simulations — one LG M50 cell discharged at 0.5C, 1C and 2C.
- Three channels at 100 Hz: current, terminal voltage, cell temperature.
- Corrupted by additive white Gaussian noise plus **impulsive outliers** — spikes, bursts and dropouts.
- The injected mask is kept on the host: the answer key, not a detector's output.

Quantity	Value
Recordings	3
Samples per recording	60 000
Channels	4
Payload	6.2 MiB
Corrupted cells	8 040
Corruption rate	**1.489%**

Noise is seeded, so regenerating reproduces the dataset exactly.

Suite	Tests	Checks
test_sensor	15	642
test_protocol	9	82
test_block_loader	8	116
test_mergesort	13	578
Total	**45**	**1418**

The same binaries build for the host and for `aarch64le`, so the suite runs in both places unchanged.

- **Boundaries** — sizes around the 4096 threshold, the partition boundaries, every size from 0 to 64.
- **Float extremes** — ± 0.0, denormals, `FLT_MIN`, `FLT_MAX`, and no NaN anywhere in the payload.
- **Determinism** — bit-identical output for 1 to 8 threads; one changed sample must be caught.
- **Real payload** — every block of real data compared against `qsort`, and the descriptor's min/max.

From source to a booting board

1. `app/Makefile` cross-compiles every program for `aarch64le` with `qcc -Vgcc_ntoaarch64le`.
2. `make install` places the binaries and the payload in the BSP's install tree.
3. `images/rpi4.build` names them for the image: seven programs into `/bin`, the payload into `/data`.
4. `mkifs` produces `ifs-rpi4.bin`; the card boots it with `kernel=ifs-rpi4.bin`.

Nothing is mounted, nothing is copied
The programs and their data live inside the image, so a board run can never pick up a stale payload — it is the image or nothing.

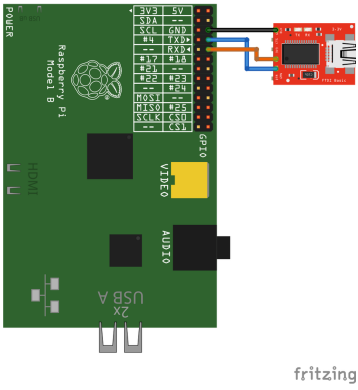
The payload is optional by design
Those build entries are marked optional, so a checkout without the generated dataset still builds a bootable image — its payload tests report skipped rather than failed. We saw both cases on the board.

PART 4

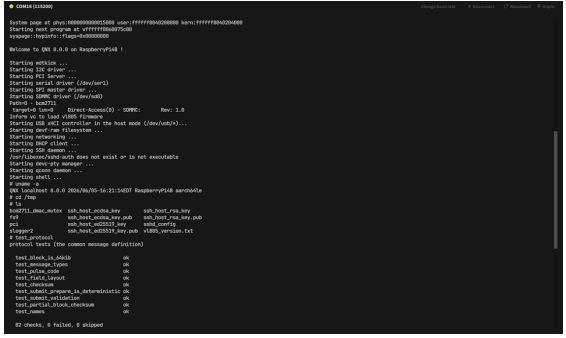
Results

Target
Raspberry Pi 4 Model B, four Cortex-A72 cores, booting QNX 8.0.0 from an SD card and driven over a serial console. No network dependency for any test, which keeps the measurements repeatable.

How we talk to it
A USB-to-serial adapter on the GPIO header: TX and RX crossed, plus ground and 3.3 V. That is the whole interface the runs below used — the board has no keyboard, display or network in the loop.

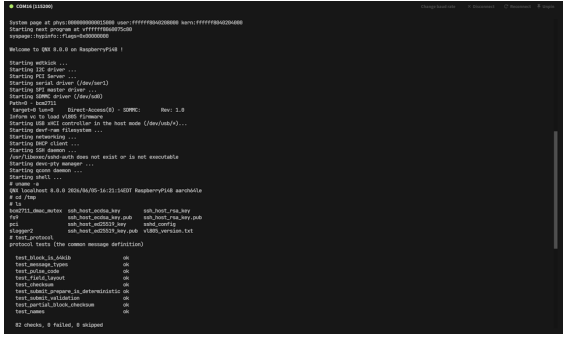


d.26 Console: the board comes up



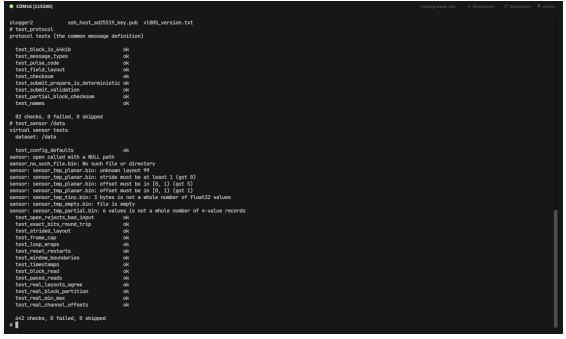
Boot to a shell. All four Cortex-A72 cores are brought up by `procnto-smp-instr`, the SD card is mounted, and the shell prompt appears — the point from which every measurement was taken.

d.27 Console: the protocol layer



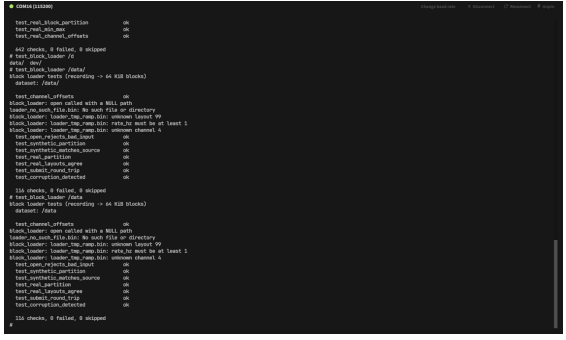
`test_protocol` — 82 checks, none failed. The 64 KiB block size, the message-type range, the pulse code, the field layout and the checksum are all pinned here, and they hold and pass on the target exactly as they do on the host.

d.28 Console: the virtual sensor



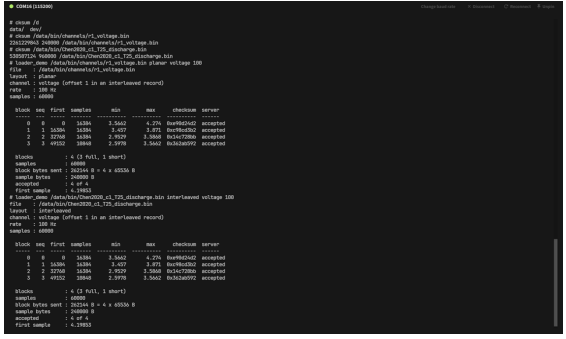
`test_sensor /data` — 642 checks, none failed. The lines above the `ok` column are the module reporting every rejected argument by name: a missing file, an unknown layout, an offset outside the stride, a file that is not a whole number of records.

d.29 Console: the block loader



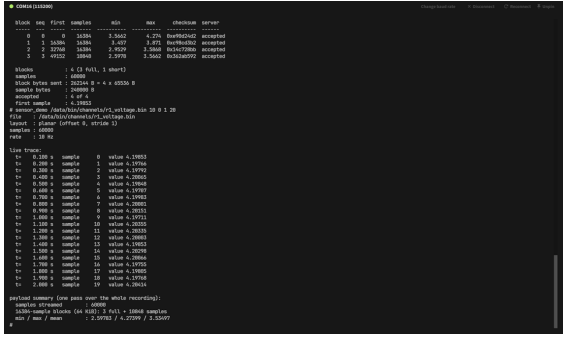
`test_block_loader /data` — 116 checks, none failed. This is the suite that proves the block partition routes produce byte-identical blocks and descriptors, and that a single changed sample is detected.

d.30 Console: the payload inside the image



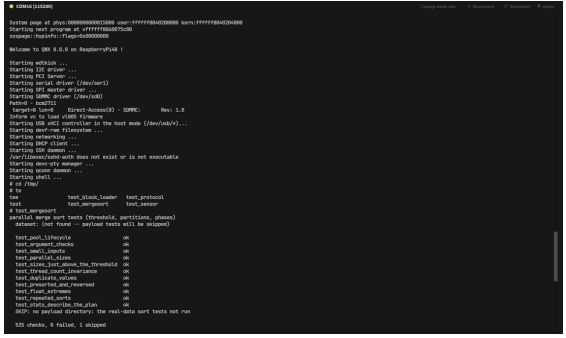
`cksum` of the payload files as they sit in the image, then `loader_demo` reading one of them. The values match the host's, so the bytes on the board are the bytes we generated — and the four block checksums show the blocks were assembled deterministically.

d.31 Console: the sensor replaying a recording



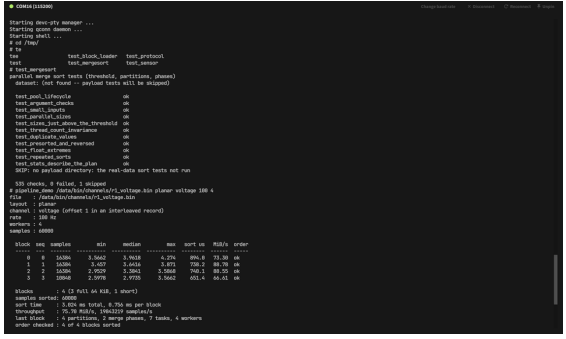
sensor_demo paced at 10 Hz — one sample at a time, on the sensor's own clock, from the payload inside the image. It then streams the whole recording: 60 000 samples in four blocks, with min, max and mean over the run.

d.32 Console: the merge sort on real data



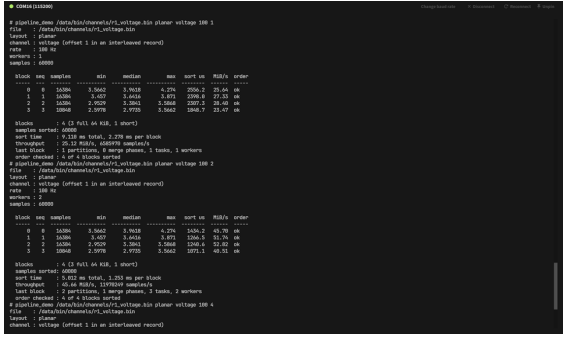
This board came up *without* the payload in its image, so the eleven unit tests ran and the two real-data tests reported skipped — 535 checks, 1 skipped, nothing failed. The skip line is the suite telling the truth rather than the image failing.

d.33 Console: sorting the payload, four workers deep



`pipeline_demo` with four workers. Each 64 KiB block is sorted, the median and range are taken from the sorted block, and the last line reports the plan: **4 partitions, 2 merge phases, 7 tasks, 4 workers**. The plan changing shape with the worker count is what shows the parallel path is doing the work.

d.34 Console: the worker sweep



The same recording at one, two and four workers. Per block: **2.278 ms** → **1.253 ms** → **0.757 ms**, which is **3.01**× on four cores. The medians, ranges and order verdicts are identical in all three runs — only the time moves.

What the numbers say

W	ms/block	MiB/s	Speed-up
1	2.278	25.12	1.00×
2	1.253	45.66	1.82×
4	**0.757**	**75.55**	**3.01×**

One 64 KiB block, 60 000-sample recording, measured on the board.

Headroom
One block is 163.84 s of signal at 100 Hz and is sorted in 0.757 ms — about **216 000×** real time.

- The plan line is fixed by n and the worker count, so it must read the same on any machine — and it does.
- **Determinism**: 4 partitions and 2 merge phases every run, with the same checksums.

d.36 What did not work, and what it cost

The short final block
60 000 samples do not divide evenly: the last block is 10 848 samples in a 16 384-slot buffer. Everything downstream must respect the descriptor's count, and the padding is what keeps the checksum from depending on stale memory.

A payload-less image
Built without the dataset, the image still boots and the suites still run; the two real-data tests report skipped. It is a build configuration, not a failure — but it has to be read correctly.

Distrust inherited from the dataset
The outlier flags in the dataset are the *injected* mask, not a detector's output. Nothing in this service claims to have found an outlier by looking at the data.

What is still open
The two programs that talk, and the filter between them. The protocol half of the notification is written and tested; the wiring is not.

PART 5

Use Cases and Closing

Where this applies

- **Battery management** — the workload came from battery recordings: continuous voltage and current, where spikes are measurement artefacts and the median is the part that survives.
- **Industrial telemetry** — a plant's sensors stream faster than a control loop can afford to sort; offloading to a worker pool keeps the collector simple.
- **Edge gateways** — a Pi-class board on a real-time kernel can sort and clean a channel's history on demand.
- **Automotive logging** — bursty diagnostic data, where a fixed 64 KiB unit and a checksum travel well over a message bus.
- **Flight and plant recorders** — the same shape: a time series that must be summarised, and an answer that must be provably the same every time it is computed.

> The pattern generalises to any stream where the work is a **fixed-size unit** and the answer must be **reproducible**.

Summary

- The unit of work is a **64 KiB block with a descriptor**, never a whole file.
- The sort is parallel, stable and **deterministic** — the answer does not depend on the machine.
- **3.01**× on four cores, 216 000× real time, measured on the board.
- **Pulse for the event, fetch for the payload** — the client never polls.
- Everything that does not need the kernel is tested off the board first: **1418** checks, clean on host and target.
- Checksums are the same across the two file layouts and across machines, which is what makes the evidence comparable.
- Next: the client and the server talking, and the median filter between them.

Questions?

THANK YOU

Real-Time Multi-Core Sorting and Median Filter Service

Anmol Govindarajapuram Krishnan - CB.EN.U4EEE23103
Mauli Rajguru - CB.EN.U4EEE23120
Sanjay TS - CB.EN.U4EEE23136

26ELC331 - Real-Time Operating Systems